Football Tournament Manager — How to Run

Ca' Foscari University of Venice · Software Architectures · A.Y. 2025–2026

Before you start

You need three things installed:

- Docker Desktop — https://www.docker.com/products/docker-desktop
- Node.js 20 or newer — https://nodejs.org
- Git — https://git-scm.com

Check they work: docker --version, node --version, git --version

1. Clone the project

```
git clone https://github.com/samantayebi/football-tournament.git
cd football-tournament
```

2. Create the .env file

```
cp .env.example .env
```

The default values work for local development. No changes needed.

3. Build the frontend

```
cd frontend
npm install
npm run build
cd ..
```

Only needed once, or when frontend code changes.

4. Start everything

```
docker compose up --build
```

Wait until you see these lines in the logs:

```
monolith-1 | Monolith listening on port 3000
gateway-1 | Configuration complete; ready for start up
```

Then open http://localhost in your browser.

First run takes a few minutes — Docker downloads the images. Add -d to run in the background.

5. Admin login

Go to http://localhost/login

```
Username: admin
Password: admin123
```

6. Basic usage flow

- Tournaments page: create a new tournament.
- Enrollment page: add teams (use the random buttons to fill in names quickly), then approve them.
- Bracket Admin: click Generate Bracket. The public bracket page now shows the draw.
- Results page: enter scores for each match (use the random button for testing). The winner advances automatically.
- After entering a result, click Add Goals to record individual goal scorers.

- Commentary page: add live text updates to a match.
- The public bracket and standings pages update automatically in any open browser tab when a result is entered — no refresh needed.

7. Run the unit tests

```
cd monolith && npm install && npm test
```

Expected: 12/12 tests pass, 100% coverage on core modules.

8. Run the load test

Install k6 first (macOS: brew install k6). Then:

```
k6 run load-test/load-test.js
```

Expected: p95 under 500ms, 0% errors, all 4 thresholds green.

> *Make sure a bracket has been generated before running, otherwise the check will fail.*

9. Stop the system

Stop containers but keep data:

```
docker compose down
```

Stop and delete all data (fresh start):

```
docker compose down -v
```

Available URLs

- http://localhost — main app
- http://localhost/login — admin login
- http://localhost/api-docs — interactive API docs (16 endpoints)
- http://localhost:3100 — Grafana dashboard (admin / admin123)
- http://localhost:9090 — Prometheus
- http://localhost:15672 — RabbitMQ management (admin / password)
- http://localhost/health/monolith/ — health check (and /standings/, /public/, /notification/)

Common problems

Port 80 already in use: something else is using port 80 (Apache, another server). Stop it first, or change 80:80 to 8080:80 in docker-compose.yml and access the app at http://localhost:8080.

Page stuck on loading: wait 30 more seconds after startup. Check curl http://localhost/health/monolith/ returns status ok.

Login fails: use admin and admin123 exactly. Check the monolith container is running with docker compose ps.

Empty bracket on public page: make sure the correct tournament is selected in the navbar dropdown.

npm install fails: check node --version is 20 or higher.
